

```
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

#define MAX_TOKENS 100
#define BUF_SIZE 512

typedef struct {
    char *value;
    int is_quoted;
} Token;

// Helper to expand environment variables like $USER or $HOME
char* expand_variable(const char *var_name) {
    char *val = getenv(var_name);
    return val ? val : "";
}

// Tokenizer handling Single and Double Quotes
int tokenize_quotes(const char *input, Token tokens[]) {
    int token_count = 0;
    int len = strlen(input);
    int i = 0;

    while (i < len) {
        // Skip leading whitespace outside quotes
        while (i < len && isspace(input[i])) i++;
        if (i >= len) break;

        char token_buf[BUF_SIZE];
        int buf_idx = 0;
        int in_single_quote = 0;
        int in_double_quote = 0;
        int is_quoted_token = 0;

        while (i < len) {
```

```

char ch = input[i];

// Handle Single Quote
if (ch == '\'' && !in_double_quote) {
    in_single_quote = !in_single_quote;
    is_quoted_token = 1;
    i++;
    continue;
}

// Handle Double Quote
if (ch == '"' && !in_single_quote) {
    in_double_quote = !in_double_quote;
    is_quoted_token = 1;
    i++;
    continue;
}

// Stop at unquoted whitespace
if (isspace(ch) && !in_single_quote && !in_double_quote) {
    break;
}

// Variable Expansion in Double Quotes or Unquoted strings
if (ch == '$' && !in_single_quote) {
    i++;
    char var_name[64];
    int var_idx = 0;
    while (i < len && (isalnum(input[i]) || input[i] == '_')) {
        var_name[var_idx++] = input[i++];
    }
    var_name[var_idx] = '\0';

    if (var_idx > 0) {
        char *expanded = expand_variable(var_name);
        while (*expanded) {
            token_buf[buf_idx++] = *expanded++;
        }
    }
}

```

```

        }
    } else {
        token_buf[buf_idx++] = '$'; // Literal '$' if no variable name follows
    }
    continue; // Skip i++ at loop end
}

// Append character literal
token_buf[buf_idx++] = ch;
i++;
}

// Error Detection: Unclosed Quotes
if (in_single_quote || in_double_quote) {
    printf("[Syntax Error] Unclosed quote detected.\n");
    return -1;
}

token_buf[buf_idx] = '\0';
tokens[token_count].value = strdup(token_buf);
tokens[token_count].is_quoted = is_quoted_token;
token_count++;
}

return token_count;

```

```

}

void free_tokens(Token tokens[], int count) {
    for (int i = 0; i < count; i++) {
        free(tokens[i].value);
    }
}

int main() {
    char input_buf[BUF_SIZE];

    while (1) {
        printf("\nQuote Parser > ");
        if (!fgets(input_buf, sizeof(input_buf), stdin)) break;

        input_buf[strcspn(input_buf, "\n")] = 0; // Strip newline

        if (strcmp(input_buf, "exit") == 0) break;

        Token tokens[MAX_TOKENS];
        int token_count = tokenize_quotes(input_buf, tokens);

        if (token_count >= 0) {
            printf("--- Token Output ---\n");
            for (int i = 0; i < token_count; i++) {
                printf("Token [%d]: \"%s\" (Quoted Mode: %s)\n",
                    i, tokens[i].value, tokens[i].is_quoted ? "YES" : "NO");
            }
            free_tokens(tokens, token_count);
        }
    }

    return 0;
}

```

OUTPUT :

```
abhis@MIKHAIL-37:~$ nano quotes_parser.c
abhis@MIKHAIL-37:~$ gcc -g -Wall quotes_parser.c -o quotes_parser
abhis@MIKHAIL-37:~$ ./quotes_parser
```

```
Quote Parser > echo '$USER is logged in'
--- Token Output ---
Token [0]: "echo" (Quoted Mode: NO)
Token [1]: "$USER is logged in" (Quoted Mode: YES)
```

```
Quote Parser > echo "$USER is logged in"
--- Token Output ---
Token [0]: "echo" (Quoted Mode: NO)
Token [1]: "abhis is logged in" (Quoted Mode: YES)
```

```
Quote Parser > echo "Hello World" 'Single Quotes: $HOME'
--- Token Output ---
Token [0]: "echo" (Quoted Mode: NO)
Token [1]: "Hello World" (Quoted Mode: YES)
Token [2]: "Single Quotes: $HOME" (Quoted Mode: YES)
```

```
Quote Parser > echo "Unclosed string
[Syntax Error] Unclosed quote detected.
```

```
Quote Parser > exit
```

```
abhis@MIKHAIL-37:~$ valgrind --leak-check=full ./quotes_parser
==2700== Memcheck, a memory error detector
==2700== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==2700== Using Valgrind-3.26.0 and LibVEX; rerun with -h for copyright info
==2700== Command: ./quotes_parser
==2700==
```

```
Quote Parser > exit
```

```
==2700==
==2700== HEAP SUMMARY:
==2700==    in use at exit: 0 bytes in 0 blocks
==2700==   total heap usage: 2 allocs, 2 frees, 2,048 bytes allocated
==2700==
==2700== All heap blocks were freed -- no leaks are possible
==2700==
==2700== For lists of detected and suppressed errors, rerun with: -s
==2700== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```